

71. JavaScript Animations

JS animations wahan kaam aate hain jahan CSS nahi pahunch pata: complex path par chalna, Bezier curves se alag timing function, ya canvas par animation.

`setInterval` se

Animation = **frames** ka sequence (HTML/CSS properties me chhote badlav). Jaise `style.left` ko `0px` se `100px` tak har kuch ms me badhana. Cinema ki tarah ~24 frames per second bhi smooth lagte hain.

```
js

let start = Date.now();

let timer = setInterval(function() {
  let timePassed = Date.now() - start;

  if (timePassed >= 2000) {
    clearInterval(timer); // 2 sec baad khatam
    return;
  }
  draw(timePassed);
}, 20);

function draw(timePassed) {
  train.style.left = timePassed / 5 + 'px'; // 0 se 400px
}
```

`requestAnimationFrame` se

Kai animations ek saath chalein aur har ek ka apna `setInterval(..., 20)` ho, to unka start time alag hota hai, isliye 20ms ke andar kai alag repaints. Behtar hai sabko **ek jagah group** karna. Saath me, CPU overloaded ho ya tab hidden ho to har 20ms chalana bekaar hai. JS me ye kaise pata ho? Iske liye **Animation timing spec** ka `requestAnimationFrame`:

```
js

let requestId = requestAnimationFrame(callback);
cancelAnimationFrame(requestId); // cancel
```

- `callback` **us waqt** chalta hai jab browser animation ke liye taiyar ho (repaint se just pehle).

- Uske andar kiye badlav dusre `requestAnimationFrame` callbacks aur CSS animations ke saath **group** ho jaate hain, to ek hi geometry recalculation aur repaint.
- `callback` ko ek argument milta hai: page load se ab tak ka time (ms), `performance.now()` jaisa.
- Aam taur par 10-20ms me chalta hai.
- Page background me ho to repaint nahi hota, to animation **ruk jaati hai** aur resources nahi khaati.

Structured animation (universal function)

```
js

function animate({timing, draw, duration}) {
  let start = performance.now();

  requestAnimationFrame(function animate(time) {
    // timeFraction 0 se 1
    let timeFraction = (time - start) / duration;
    if (timeFraction > 1) timeFraction = 1;

    let progress = timing(timeFraction); // animation ki current state
    draw(progress); // draw karo

    if (timeFraction < 1) requestAnimationFrame(animate);
  });
}
```

3 parameters animation ko describe karte hain:

- `duration`: total time (ms), jaise `1000`.
- `timing(timeFraction)`: CSS `transition-timing-function` jaisa. Bitra hua time fraction (0 se 1) leta hai aur animation ka progress (Bezier me `y`) jaisa) lautata hai.

```
js

function linear(timeFraction) { return timeFraction; }
```

- `draw(progress)`: progress ko draw karta hai. `0` = shuruaati state, `1` = end state.

```
js
```

```
animate({
  duration: 1000,
  timing(timeFraction) { return timeFraction; },
  draw(progress) { elem.style.width = progress * 100 + '%'; }
});
```

CSS ke ulta yaha **koi bhi timing function** aur koi bhi draw function bana sakte ho. Timing Bezier tak limited nahi, aur **draw** sirf properties nahi, naye elements bana kar (jaise fireworks) bhi kuch kar sakta hai.

Timing functions

Power of n (tez hone wali):

```
js
function quad(timeFraction) { return Math.pow(timeFraction, 2); }
```

n jitna bada, utna tez speed up (cubic, **n=5** waghera).

Arc:

```
js
function circ(timeFraction) { return 1 - Math.sin(Math.acos(timeFraction)); }
```

Back (teer chalana): pehle string kheenchna, phir chhodna. "Elasticity coefficient" **x** leta hai.

```
js
function back(x, timeFraction) {
  return Math.pow(timeFraction, 2) * ((x + 1) * timeFraction - x);
}
```

(x = 1.5) example)

Bounce: gend girti hai, kai baar uchhalti hai, rukti hai. Ye function ulte order me: bouncing turant shuru.

```
js
```

```
function bounce(timeFraction) {
  for (let a = 0, b = 1; 1; a += b, b /= 2) {
    if (timeFraction >= (7 - 4 * a) / 11) {
      return -Math.pow((11 - 6 * a - 11 * timeFraction) / 4, 2) + Math.pow(b, 2)
    }
  }
}
```

Elastic: "initial range" (x) leta hai.

```
js

function elastic(x, timeFraction) {
  return Math.pow(2, 10 * (timeFraction - 1)) * Math.cos(20 * Math.PI * x / 3 *
```

Reversal: (ease*)

In functions ko seedha lagana **easeIn** kehlata hai. Ulta dikhane ke liye **easeOut**.

easeOut: (timingEaseOut(timeFraction) = 1 - timing(1 - timeFraction))

```
js

function makeEaseOut(timing) {
  return function(timeFraction) {
    return 1 - timing(1 - timeFraction);
  };
}

let bounceEaseOut = makeEaseOut(bounce); // bounce ab end me hoga, shuru me n
```

Regular bounce: object neeche bounce karta hai, end me achanak upar. (easeOut) ke baad: pehle upar jaata hai, phir wahin bounce.

easeInOut: effect shuru aur end dono me:

```
js
```

```
function makeEaseInOut(timing) {
  return function(timeFraction) {
    if (timeFraction < .5)
      return timing(2 * timeFraction) / 2;
    else
      return (2 - timing(2 * (1 - timeFraction))) / 2;
  };
}

bounceEaseInOut = makeEaseInOut(bounce);
```

Pehla aadha hissa chhota kiya hua `easeIn`, doosra aadha chhota kiya hua `easeOut`.

Aur interesting `draw`

Element hilane ke alawa kuch bhi. Jaise "bouncing" text typing animation:

```
js

function animateText(textArea) {
  let text = textArea.value;
  let to = text.length, from = 0;

  animate({
    duration: 5000,
    timing: bounce,
    draw: function(progress) {
      let result = (to - from) * progress + from;
      textArea.value = text.slice(0, Math.ceil(result));
    }
  });
}
```

Summary

- CSS se jo na ho ya tight control chahiye, uske liye JS animation, `requestAnimationFrame` se.
- Page background me ho to callbacks nahi chalte, animation ruk jaati hai aur resources bachte hain.
- Helper `animate({timing, draw, duration})` zyaadatar animations set up kar deta hai.
- Koi bhi timing function chal sakta hai, aur `draw` se kuch bhi animate kar sakte ho.
- JS animations roz nahi lagte, kuch interesting aur non-standard ke liye. Zarurat ke hisaab se features jodo.

Tasks

- **Bouncing ball:** ball (position:absolute) (field (position:relative)), (top) ko (0) se (field.clientHeight - ball.clientHeight) tak.

js

```
let to = field.clientHeight - ball.clientHeight;
animate({
  duration: 2000,
  timing: makeEaseOut(bounce),
  draw(progress) { ball.style.top = to * progress + 'px'; }
});
```

- **Ball bouncing to the right:** upar wala + ek aur (animate) (left) ke liye (bounce nahi, dheere badhta hua), (makeEaseOut(quad)) achha lagta hai:

js

```
animate({
  duration: 2000,
  timing: makeEaseOut(quad),
  draw(progress) { ball.style.left = 100 * progress + "px"; }
});
```